

Entrega 1

Proyecto final

Análisis y diseño de algoritmos

Índice

Definición del problema:	2
Entradas:	2
Salidas:	3
Restricciones:	3
Solución Fuerza bruta	4
Análisis de complejidad — Fuerza Bruta:	4
Gráfica comparativa (tiempo vs tamaño de entrada):.....	5
Análisis:	5
Conclusión:	5

Definición del problema:

Este proyecto aborda un problema sobre un sistema de logística y entregas de una empresa de paquetería. La empresa cuenta con una flota de camiones encargados de transportar paquetes hacia diferentes destinos, teniendo en cuenta restricciones de capacidad y tiempos de entrega.

El problema consiste en determinar y asignar eficientemente paquetes a camiones, buscando maximizar la cantidad de entregas realizadas sin exceder la capacidad de carga de los vehículos y respetando las restricciones temporales de cada paquete.

Este tipo de problema es común en sistemas reales de distribución, comercio electrónico y cadenas de suministro, donde una mala planificación puede generar retrasos, sobrecostos y baja eficiencia operativa.

Entradas:

Paquetes (n paquetes)

Con la siguiente información:

Campo	Descripción
ID	Identificador único del paquete
Peso	Peso del paquete
Destino	Lugar de entrega
Tiempo límite	Tiempo máximo permitido para la entrega
Prioridad	Nivel de prioridad del paquete

Ejemplo: 1, 10, A, 5, 1

Representa:

- Paquete 1
- Peso: 10
- Destino: A
- Tiempo límite: 5
- Prioridad: 1

Camiones (m camiones)

Campo	Descripcion
ID	Identificador del camión
Capacidad	Peso máximo soportado

Ejemplo: 101, 40

Representa:

- Camión 101
- Capacidad máxima: 40

Salidas:

La salida esperada del sistema es una asignación de paquetes a camiones que indique:

- Qué paquetes serán transportados por cada camión
- Que ninguna capacidad sea excedida
- El conjunto de entregas realizadas

Restricciones:

El problema debe cumplir las siguientes restricciones:

1. La suma de pesos de los paquetes asignados a un camión no puede superar su capacidad máxima.
2. Cada paquete puede asignarse como máximo a un solo camión.
3. Deben considerarse las restricciones temporales de entrega.
4. Se busca maximizar la eficiencia de distribución y la cantidad de paquetes entregados.

Solución Fuerza bruta

Análisis de complejidad — Fuerza Bruta:

Complejidad temporal: $O((m + 1)^n)$

La solución explora todas las formas posibles de asignar n paquetes a m camiones. Para cada paquete hay $m + 1$ opciones (los m camiones más la opción de no asignarlo), y esto se repite para los n paquetes, lo que genera $(m + 1)^n$ combinaciones en total.

Por ejemplo:

- Con 5 paquetes y 2 camiones (caso pequeño) se exploran $3^5 = 243$ operaciones.
- Con 15 paquetes y 4 camiones (caso mediano) se exploran $5^{15} = 30.517.578.125$ operaciones.
- Con 40 paquetes y 8 camiones (caso grande) se exploran 9^{40} operaciones, un número tan grande que es computacionalmente inviable, es por eso por lo que el sistema no arroja ningún resultado.

Complejidad espacial: $O(n * m)$

En memoria se almacenan la asignación actual (una lista por camión con los paquetes asignados) y las cargas actuales de cada camión. El espacio máximo usado en cualquier momento es proporcional al número de paquetes por el número de camiones.

Justificación paso a paso:

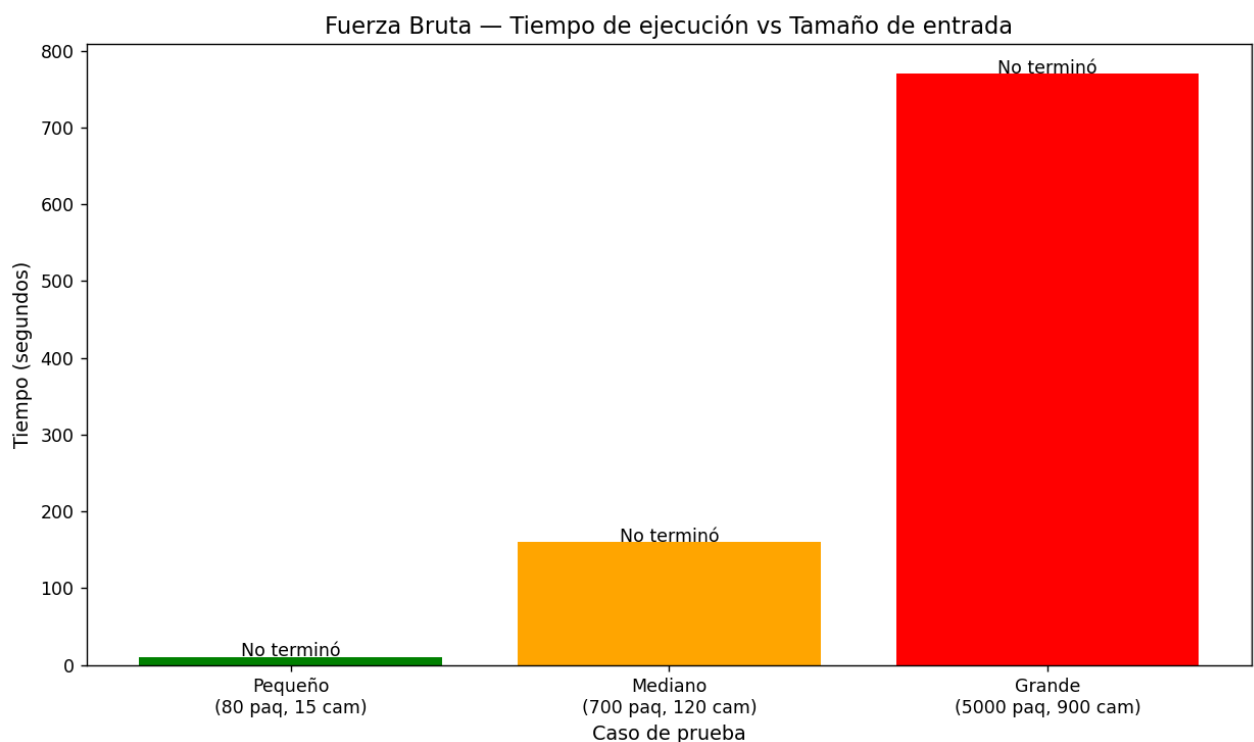
1. $product(range(m + 1), repeat = n)$ genera $(m + 1)^n$ combinaciones en total
2. Por cada combinación se itera sobre los n paquetes para validar capacidad: $O(n)$
3. Por cada combinación válida se recorren los m camiones para guardar el resultado: $O(m)$
4. Total por combinación: $O(n + m)$

Medición empírica de tiempos con diferentes tamaños de entrada:

Caso	Paquetes	Camiones	Combinaciones $(m+1)^n$	Tiempo real
Pequeño	80	15	$16^{80} \approx \dots$	No termina
Mediano	700	120	$121^{700} \approx \dots$	No termina
Grande	5000	900	$901^{5000} \approx \dots$	No termina

La fuerza bruta solo es ejecutable en el caso pequeño. A medida que aumentan los paquetes y camiones el tiempo crece de forma exponencial, haciendo imposible su uso en casos reales. Así se justifica la necesidad de usar algoritmos más eficientes como greedy, backtracking y divide y vencerás para los casos mediano y grande que veremos más adelante.

Gráfica comparativa (tiempo vs tamaño de entrada):



Análisis:

La diferencia en tiempos se explica directamente por la complejidad de cada algoritmo. La fuerza bruta tiene complejidad $O((m + 1)^n)$, lo que la hace inviable para casos medianos y grande

Conclusión:

No existe un algoritmo perfecto, siempre hay factores buenos y malos entre que haya velocidad y sea óptimo. La fuerza bruta es óptima pero no escala. La recursiva escala, pero sacrifica calidad de solución. Para los siguientes algoritmos (greedy, backtracking, divide y vencerás) esperamos que haya un equilibrio entre ambos aspectos.